

Week 1 report of Internship

Name Ammar Akram

ID DVS-8041

Domain Machine Learning

Office Devsphere

Date 5/09/2026

DEVSPHERE AI & ML INTERNSHIP

PYTHON + PANDAS DATA ANALYSIS:

PROJECT TITLE:

STUDENT PERFORMANCE DATA ANALYSIS AND CLEANING

Week: 1

Task: Perform Data Analysis on a Dataset

Technology: Python, Pandas and Matplotlib

Submitted as part of the DevSphere Internship Program

Deadline: 6 September 2026

INTRODUCTION:

This project was completed as part of the DevSphere AI & ML Internship Week 1 task. The main purpose of the task was to work with a real student performance dataset and understand how Python and Pandas can be used for data analysis and data cleaning.

The work started with loading the Mathematics student performance dataset and checking whether it was read correctly. After that, the structure of the dataset, data types, summary statistics, missing values, duplicate records, categorical values, and numerical values were examined.

A separate copy of the original dataset was then created for cleaning. The original dataset was kept unchanged so that the raw data would remain available for reference. After validation, the cleaned dataset was exported as a new CSV file.

The final part of the task focused on basic analysis of student grades. The G1, G2, and G3 columns were analyzed, and a histogram was created to understand the distribution of final mathematics grades.

This project gave practical experience with a complete basic data-analysis workflow, starting from raw data and ending with a validated and documented dataset.

OBJECTIVES:

The main objectives of this project were:

- Load the Student Performance dataset using Pandas.
- Inspect the first five records of the dataset.
- Understand the number of records and columns.
- Check the dataset structure and data types.
- Generate numerical and categorical summary statistics.
- Identify missing values.
- Check for duplicate records.
- Inspect important categorical values.
- Check numerical ranges for potentially invalid values.
- Create a separate cleaned copy of the dataset.
- Validate the cleaned data.
- Export the cleaned dataset as a CSV file.
- Perform basic analysis on student grades.
- Compare average final grades between gender groups.
- Create a visualization of final mathematics grade distribution.
- Document the complete process using code, outputs, and screenshots.

DATASET DESCRIPTION:

The dataset used for this project is the Student Performance dataset, specifically the Mathematics student performance data.

The dataset contains information about students' demographic background, family and social characteristics, school-related information, and academic performance.

The dataset contains 395 student records and 33 columns.

The three main academic performance columns used during the analysis were:

G1 — First period grade

G2 — Second period grade

G3 — Final grade

The original CSV file uses a semicolon (;) as the separator between values. Therefore, the separator was specified while loading the dataset.

The original dataset was not modified. A separate copy was created for the cleaning and analysis process.

TOOLS AND TECHNOLOGIES:

The following tools were used during the project:

- Python 3.13.15
- Python was used as the main programming language for loading, cleaning, analyzing, and processing the dataset.
- Pandas
- Pandas was used to read the CSV file, inspect the dataset, perform data-quality checks, calculate statistics, group data, and export the cleaned dataset.
- Matplotlib
- Matplotlib was used to create a histogram showing the distribution of final mathematics grades.
- Jupyter Notebook
- Jupyter Notebook was used to run the analysis step by step and display the code and outputs together.
- Visual Studio Code
- Visual Studio Code was used to work with the Python source code and project files.
- Microsoft Word
- Microsoft Word was used to prepare the final project report.

METHODOLOGY:

The project followed a simple step-by-step workflow.

Step 1 — Load the Dataset

The Student Performance Mathematics dataset was loaded into a Pandas DataFrame.

Step 2 — Initial Inspection

The first five records were displayed to make sure the data had been loaded correctly.

Step 3 — Dataset Information

The number of rows, columns, data types, and general structure of the dataset were inspected.

Step 4 — Summary Statistics

Numerical and categorical summary statistics were generated.

Step 5 — Data Quality Checks

The dataset was checked for missing values, duplicate records, data types, categorical values, and numerical ranges.

Step 6 — Data Cleaning

A separate copy of the original dataset was created. Cleaning was performed only when necessary.

Step 7 — Validation

The cleaned dataset was checked again to confirm that it remained complete and consistent.

Step 8 — Export

The cleaned dataset was saved as a separate CSV file.

Step 9 — Basic Analysis

The G1, G2, and G3 grade columns were analyzed using basic statistical calculations.

Step 10 — Visualization

A histogram was created to show the distribution of final mathematics grades.

DATASET LOADING AND INITIAL INSPECTION:

6.1 Loading the Dataset

The dataset was loaded using Pandas' `read_csv()` function.

Because the original file uses semicolons as separators, `sep=";"` was used.

Code:

```
import pandas as pd  
df = pd.read_csv("../data/student-mat.csv", sep=";")
```

6.2 Checking the First Five Records

After loading the dataset, the first five rows were displayed.

Code:

```
print(df.head())
```

Output:

The command displayed the first five student records along with the dataset columns. This confirmed that the CSV file was loaded correctly.

The dataset begins with student information such as school, gender, age, address, family information, study-related factors, and academic grades.

Figure 1 — Dataset Preview:

```
[3]: # Display the first five rows of the dataset
df.head()
```

```
[3]:
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	3	6	5	6	6
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	3	4	5	5	6
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	3	10	7	8	10
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	5	2	15	14	15
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	5	4	6	10	10

5 rows × 33 columns

```
[4]: # Display dataset information
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
#   Column      Non-Null Count  Dtype
---  -
0   school      395 non-null    str
1   sex         395 non-null    str
2   age         395 non-null    int64
```

6.3 Checking Dataset Size:

The shape of the dataset was checked using:

Code:

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Output:

Rows: 395
Columns: 33

This confirmed that the Mathematics dataset contains 395 student records and 33 columns.

DATASET INFORMATION AND SUMMARY STATISTICS

7.1 Dataset Information:

The info() function was used to understand the structure of the DataFrame.

Code:

```
df.info()
```

Output:

The dataset contains 395 entries and 33 columns.

The columns contain both numerical and categorical information. The numerical columns include student age, education levels, study-related values, absences, and grades, while several columns contain categorical information.

The dataset uses integer and object/string-based columns.

The information output also confirmed that there were no null values in the dataset.

```
[5]: # Generate summary statistics for numerical columns
df.describe()
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	Walc	health	abser
count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177	3.944304	3.235443	3.108861	1.481013	2.291139	3.554430	5.708101
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651	0.896659	0.998862	1.113278	0.890741	1.287897	1.390303	8.003101
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.000000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000	3.000000	2.000000	1.000000	1.000000	3.000000	0.000000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000	3.000000	3.000000	1.000000	2.000000	4.000000	4.000000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000	4.000000	4.000000	2.000000	3.000000	5.000000	8.000000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000	5.000000	5.000000	5.000000	5.000000	5.000000	75.000000

```
[6]: # Summary statistics for categorical columns
df.describe(include="object")
```

7.2 Numerical Summary Statistics:

The describe() function was used to calculate statistical information for numerical columns.

Code:

```
print(df.describe())
```

Output:

The summary included count, mean, standard deviation, minimum, percentile values, and maximum values.

For the grade columns, the main results were:

G1 average: approximately 10.91

G2 average: approximately 10.71

G3 average: approximately 10.42

G3 minimum: 0

G3 maximum: 20

G3 median: 11

7.3 Categorical Summary Statistics:

Categorical columns were also examined.

Code:

```
print(df.describe(include="object"))
```

Output:

The output provided information such as the number of unique values, most common value, and frequency for categorical columns.

This helped in understanding the general distribution of non-numerical information in the dataset.

```
[5]: # Generate summary statistics for numerical columns
df.describe()
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	Walc	health	abser
count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177	3.944304	3.235443	3.108861	1.481013	2.291139	3.554430	5.708
std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651	0.896659	0.998862	1.113278	0.890741	1.287897	1.390303	8.003
min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	0.000
25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000	4.000000	3.000000	2.000000	1.000000	1.000000	3.000000	0.000
50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000	4.000000	3.000000	3.000000	1.000000	2.000000	4.000000	4.000
75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000	5.000000	4.000000	4.000000	2.000000	3.000000	5.000000	8.000
max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000	5.000000	5.000000	5.000000	5.000000	5.000000	5.000000	75.000

```
[6]: # Summary statistics for categorical columns
df.describe(include="object")
```

DATA QUALITY ANALYSIS:

Before making any changes to the dataset, several data-quality checks were performed.

8.1 Missing Values:

The dataset was checked for missing values using Pandas.

Code:

```
missing_values = df.isnull().sum()
print(missing_values)
```

Output:

All columns returned 0 missing values.

This means that no missing-value treatment was required for the dataset.

A total missing-value check was also performed:

Code:

```
print("Total missing values:", df.isnull().sum().sum())
```

Output:

Total missing values: 0

8.2 Duplicate Records:

The dataset was checked for duplicate rows.

Code:

```
print("Duplicate rows:", df.duplicated().sum())
```

Output:

Duplicate rows: 0

No duplicate records were found.

8.3 Data Types:

The data types of the columns were inspected.

Code:

```
print(df.dtypes)
```

Output:

The output showed that the dataset contains both numerical and categorical columns.

The numerical columns are represented using integer data types, while categorical fields such as school, gender, address, family size, and occupations are represented as object/string data.

8.4 Checking Important Categorical Values:

Important categorical columns were inspected to make sure that their values were reasonable.

Code:

```
categorical_columns = ["school", "sex", "address", "famsize", "Pstatus"]
```

```
for column in categorical_columns:
```

```
print(column, ":", df[column].unique())
```

Output:

The categorical columns contained expected values such as:

school: GP, MS

sex: F, M

address: U, R

famsize: LE3, GT3

Pstatus: A, T

8.5 Numerical Range Checks:

Numerical columns were inspected for their minimum and maximum values.

Code:

```
numeric_columns = df.select_dtypes(include="number")

print("Minimum values:")
print(numeric_columns.min())

print("\nMaximum values:")
print(numeric_columns.max())
```

Output:

The numerical values were reviewed to identify potentially invalid or unexpected ranges.

The grade columns G1, G2, and G3 use a 0–20 grading scale, with G3 ranging from 0 to 20.

```
[8]: # Calculate the total number of missing values
total_missing = df.isnull().sum().sum()

print("Total missing values:", total_missing)

Total missing values: 0

[9]: # Check for duplicate records
duplicate_count = df.duplicated().sum()

print("Number of duplicate rows:", duplicate_count)

Number of duplicate rows: 0

[10]: # Display the data type of each column
df.dtypes

[10]: school      str
sex             str
age            int64
address        str
famsize        str
Pstatus        str
Medu          int64
Fedu          int64
Mjob          str
Fjob          str
reason        str
guardian      str
traveltime    int64
...
```

DATA CLEANING:

The original dataset was preserved before performing any cleaning operations.

A separate DataFrame was created:

Code:

```
cleaned_df = df.copy()
```

This approach ensured that the original raw dataset remained unchanged.

9.1 Missing-Value Validation

The cleaned DataFrame was checked for missing values.

Code:

```
print("Missing values after cleaning:",  
      cleaned_df.isnull().sum().sum())
```

Output:

Missing values after cleaning: 0

9.2 Duplicate Validation

Duplicate records were checked again.

Code:

```
print("Duplicate rows after cleaning:",  
      cleaned_df.duplicated().sum())
```

Output:

Duplicate rows after cleaning: 0

9.3 Cleaning Decision

The quality checks showed that the original dataset already contained:

Missing values: 0

Duplicate records: 0

Because there were no missing values or duplicate records, no valid rows were unnecessarily removed.

This was an important part of the cleaning process. Data cleaning does not always mean deleting or changing data. When the data is already complete and consistent, the correct approach is to validate it and preserve the valid records.

Columns: 33

Changes:

Rows removed: 0

Columns removed: 0

```
[24]: # Verify the exported cleaned dataset

verified_df = pd.read_csv("../data/student-mat-cleaned.csv")

print("Exported file verified successfully.")
print("Shape:", verified_df.shape)
print("Missing values:", verified_df.isnull().sum().sum())
print("Duplicate rows:", verified_df.duplicated().sum())

Exported file verified successfully.
Shape: (395, 33)
Missing values: 0
Duplicate rows: 0
```

CLEANED DATASET EXPORT AND VERIFICATION:

After the cleaning and validation process, the cleaned DataFrame was exported to a separate CSV file.

10.1 Exporting the Cleaned Dataset

Code:

```
cleaned_df.to_csv(
    "../data/student-mat-cleaned.csv",
    index=False
)
```

Output:

The cleaned dataset was successfully saved as:

student-mat-cleaned.csv

10.2 Loading the Exported File

To make sure that the saved file could be read correctly, it was loaded again.

Code:

```
verified_df = pd.read_csv(
    "../data/student-mat-cleaned.csv"
)

print("Verified shape:", verified_df.shape)
print("Missing values:",
    verified_df.isnull().sum().sum())
print("Duplicate rows:",
    verified_df.duplicated().sum())
```

Output:

Verified shape: (395, 33)

Missing values: 0

Duplicate rows: 0

This verification confirmed that the exported cleaned dataset retained all 395 records and 33 columns and did not introduce missing or duplicate records.

```
dtype: object

[22]: # Export the cleaned dataset
output_path = "../data/student-mat-cleaned.csv"

cleaned_df.to_csv(output_path, index=False)

print("Cleaned dataset exported successfully.")
print("Saved to:", output_path)

Cleaned dataset exported successfully.
Saved to: ../data/student-mat-cleaned.csv
```

BASIC DATA ANALYSIS:

After cleaning the dataset, basic analysis was performed on student academic performance.

The main columns used were:

G1 — First period grade

G2 — Second period grade

G3 — Final grade

11.1 Average Grade Analysis:

The mean of each grade column was calculated.

Code:

```
print("Average G1:", cleaned_df["G1"].mean())
print("Average G2:", cleaned_df["G2"].mean())
print("Average G3:", cleaned_df["G3"].mean())
```

Output:

Average G1: approximately 10.91

Average G2: approximately 10.71

Average G3: approximately 10.42

The results show that the average final grade was slightly lower than the averages of the first and second period grades.

11.2 Final Grade Statistics

The final grade was examined in more detail.

Code:

```
print("G3 Mean:", cleaned_df["G3"].mean())  
print("G3 Median:", cleaned_df["G3"].median())  
print("G3 Minimum:", cleaned_df["G3"].min())  
print("G3 Maximum:", cleaned_df["G3"].max())
```

Output:

G3 Mean: approximately 10.42

G3 Median: 11.0

G3 Minimum: 0

G3 Maximum: 20

This shows that the final mathematics grade ranges from 0 to 20, with a median of 11.

11.3 Average Final Grade by Gender

The average final grade was also compared between the gender groups.

Code:

```
gender_analysis = cleaned_df.groupby("sex")["G3"].mean()  
print(gender_analysis)
```

Output:

The output displayed the average final mathematics grade separately for female and male students.

This comparison was used only to describe the data. It does not indicate that gender causes differences in academic performance.

DATA VISUALIZATION:

A histogram was created to visualize the distribution of final mathematics grades.

Code:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.hist(
    cleaned_df["G3"],
    bins=10
)

plt.title("Distribution of Final Mathematics Grades")
plt.xlabel("Final Grade (G3)")
plt.ylabel("Number of Students")

plt.show()
```

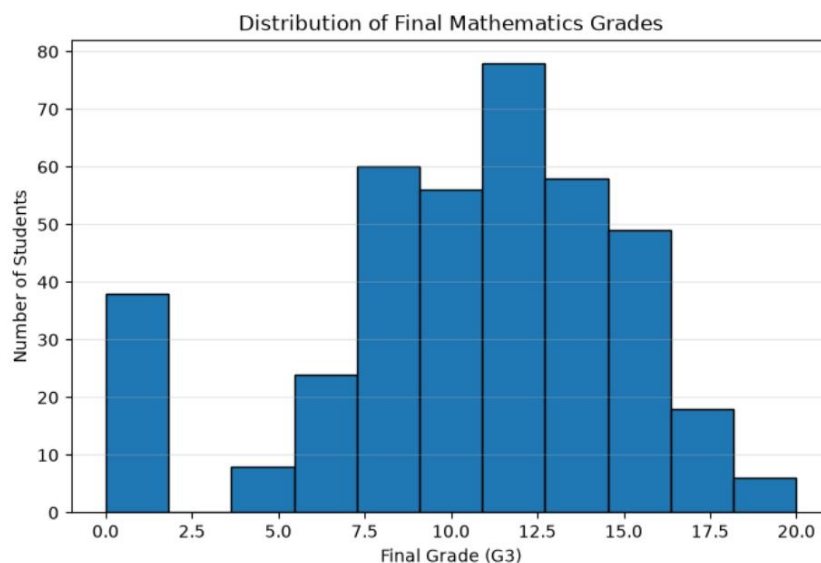
Output:

The histogram shows how the final mathematics grades are distributed among the 395 students.

The visualization makes it easier to see which grade ranges contain more students and how the grades are spread across the 0–20 scale.

```
plt.ylabel("Number of Students")
plt.grid(axis="y", alpha=0.3)

plt.show()
```



FINDINGS:

- The project produced several useful findings.
- First, the Student Performance Mathematics dataset contains 395 student records and 33 columns.
- The dataset was successfully loaded using Pandas, and its first records were inspected.
- The dataset structure and data types were examined successfully.
- The quality checks found zero missing values.
- The duplicate check also found zero duplicate records.
- Important categorical values were inspected and found to contain expected categories.
- Numerical ranges were reviewed, including the 0–20 range of the grade columns.
- Because there were no missing values or duplicate records, no valid records needed to be removed.
- A separate cleaned copy of the dataset was created to protect the original data.
- The cleaned dataset was exported successfully as student-mat-cleaned.csv.
- The exported file was loaded again and verified successfully.
- Basic grade analysis showed average values of approximately 10.91 for G1, 10.71 for G2, and 10.42 for G3.
- The final grade G3 had a minimum value of 0, a maximum value of 20, and a median of 11.
- Average final grades were also compared between gender groups.
- Finally, a histogram was created to visualize the distribution of final mathematics grades.

CONCLUSION:

This Week 1 project provided practical experience in using Python and Pandas for data analysis and data cleaning.

The project followed a complete workflow starting from loading the raw dataset and inspecting its contents, followed by checking data quality, creating a cleaned copy, exporting the result, and performing basic analysis.

One of the main results of the quality checks was that the dataset did not contain missing values or duplicate records. Because of this, no unnecessary records were deleted. Instead, the data was validated and preserved.

The analysis of G1, G2, and G3 provided a basic view of student academic performance. The final grade was also examined using mean, median, minimum, and maximum values.

A histogram was created to make the distribution of final grades easier to understand.

Overall, the project helped build practical experience with the basic data-analysis process and showed how Pandas can be used to prepare and analyze a real dataset in a structured way. The cleaned dataset is now ready for further analysis or future machine learning work.

PROJECT STRUCTURE:

The project was organized into separate folders to keep the data, code, notebook, screenshots, and report easy to find.

DevSphere_ML_Week1

- |
- | — data
 - | | — student-mat.csv
 - | | — student-mat-cleaned.csv
- |
- | — notebook
 - | | — DevSphere_ML_Week1_Analysis.ipynb
- |
- | — screenshots
 - | | — 01_load_and_preview.png
 - | | — 02_dataset_info_statistics.png
 - | | — 03_data_quality_analysis.png
 - | | — 04_data_cleaning.png
 - | | — 05_export_verification.png
 - | | — 06_analysis_visualization.png
- |
- | — report
 - | | — DevSphere_ML_Week1_Report.docx
 - | | — DevSphere_ML_Week1_Report.pdf
- |
- | — src
 - | | — data_analysis.py
- |
- | — README.md